

Replikation und Konsens

Worum geht es?

In verteilten Systemen werden Daten oft **mehrfach gespeichert**. Das nennt man **Replikation**.

Ziele dabei:

- **Fehlertoleranz:** Fällt ein Knoten aus, bleiben Daten über andere Replikas verfügbar
- **Lastverteilung:** Zugriffe können auf mehrere Replikas verteilt werden
- **Nähe zum Nutzer:** Daten können geografisch verteilt liegen

Das Problem beginnt, sobald sich Daten **ändern**. Dann müssen mehrere Replikas **konsistent** gehalten werden.

1. Grundproblem der Replikation

Statische Daten

Wenn sich Daten nicht ändern, ist Replikation einfach:

- Daten kopieren
- Fertig

Dynamische Daten

Wenn Clients schreiben, liken, löschen oder aktualisieren, entstehen Probleme:

- Nachrichten können verloren gehen
 - Requests können doppelt ankommen
 - Replikas können unterschiedliche Zustände haben
 - Gleichzeitige Updates können kollidieren
-

2. Idempotenz und Wiederholungen

Idempotenz

Eine Operation ist **idempotent**, wenn mehrfaches Ausführen denselben Effekt hat wie einmaliges Ausführen.

Beispiel

- **Nicht idempotent:** `likeCount = likeCount + 1`
- **Idempotent:** `likeSet = likeSet ∪ {userID}`

Das ist wichtig, weil Clients oder Server Requests oft **wiederholen**, wenn keine Bestätigung kommt.

Zustell-Semantiken

Only-once

- Anfrage wird nur einmal gesendet
- Kann verloren gehen

At-least-once

- Anfrage wird wiederholt, bis eine Bestätigung kommt
- Kann mehrfach ausgeführt werden

Exactly-once

- Anfrage wird wiederholt
 - Duplikate werden verhindert oder unschädlich gemacht
 - Dafür braucht man Idempotenz oder Duplikaterkennung
-

3. Typische Konflikte bei Replikation

Problem: Add und Remove

Operationen wie

- `add(x)`
- `remove(x)`

sind einzeln idempotent, aber in Kombination problematisch.
Je nach Reihenfolge kann ein anderer Endzustand entstehen.

Problem: Replikas sehen verschiedene Reihenfolgen

Zwei Replikas können dieselben Updates in unterschiedlicher Reihenfolge sehen.
Dann können sie nicht mehr sicher unterscheiden, was der „richtige“ Endzustand ist.

4. Lösungsidee: Zeitstempel und Tombstones

Tombstones

Beim Löschen wird ein Wert nicht wirklich entfernt.
Stattdessen wird markiert:

- Wert existiert
- aber ist „unsichtbar“ / gelöscht

So ein Marker heißt **Tombstone** oder **Grabstein**.

Logische Zeitstempel

Jeder Eintrag bekommt zusätzlich einen **logischen Zeitstempel**, der die letzte Schreiboperation beschreibt.

Beispiel:

- `(t1, add(x))`
- `(t2, remove(x))`

Wenn `t2 > t1`, ist der spätere Zustand maßgeblich.

5. Anti-Entropy

Replikas tauschen periodisch ihren Zustand aus, um Inkonsistenzen zu reparieren.

Vorgehen:

- Replikas vergleichen Metadaten
- Für jede Variable bleibt die Version mit dem **neueren Zeitstempel**
- Ältere Zustände werden verworfen

Das nennt man **Anti-Entropy**.

6. Nebenläufige Schreibvorgänge

Wenn zwei Clients fast gleichzeitig unterschiedliche Werte schreiben, gibt es Konflikte.

Strategie 1: Last Writer Wins (LWW)

- Der Schreibzugriff mit dem „größeren“ Zeitstempel gewinnt
- Einfach

- Nachteil: mögliche **Datenverluste**

Strategie 2: Multi-Value Register

- Wenn zwei Schreibzugriffe nebenläufig sind, bleiben **beide Werte erhalten**
- Erst später muss aufgelöst werden, welcher gilt

Diese konkurrierenden Werte nennt man auch:

- **Conflicts**
 - **Siblings**
-

7. Quoren

Idee

Nicht immer müssen alle Replikas antworten.
Stattdessen reicht eine **Mindestmenge**.

Das nennt man **Quorum**.

Write Quorum und Read Quorum

In einem System mit n Replikas:

- Schreibzugriff muss von w Replikas bestätigt werden
- Lesezugriff liest von r Replikas

Wenn gilt:

$$r + w > n$$

dann überschneiden sich Lese- und Schreibmenge immer in mindestens einem Knoten.
Damit sieht eine Leseoperation mindestens einen aktuellen Wert.

Read-after-write Consistency

Ziel:

Wenn ein Client etwas schreibt und direkt danach liest,
soll er mindestens seinen eigenen Wert oder einen noch neueren sehen.

Das erreicht man mit geeigneten Read-/Write-Quoren.

Read Repair

Wenn beim Lesen auffällt, dass einige Replikas älter sind:

- Client oder Server erkennt die neueste Version
- diese wird an veraltete Replikas zurückverteilt

So werden Inkonsistenzen beim Lesen repariert.

8. Replikation mit Broadcast

Je stärker der Broadcast-Mechanismus, desto weniger Annahmen braucht die Anwendung.

Zusammenhang zwischen Broadcast und Semantik

- **Total Order Broadcast**: reicht für deterministische Zustandsmaschinen
 - **Causal Broadcast**: reicht, wenn nebenläufige Updates vertauschbar sind
 - **Reliable Broadcast**: reicht, wenn alle Updates kommutativ sind
 - **Best-effort Broadcast**: nur bei idempotenten, kommutativen Updates, die sogar Nachrichtenverlust verkraften
-

9. State Machine Replication (SMR)

Grundidee

Alle Replikas erhalten **dieselben Nachrichten in derselben Reihenfolge**.

Jeder Replika wendet diese dann mit **deterministischer Logik** auf seinen lokalen Zustand an.

Wenn alle mit demselben Startzustand und derselben Reihenfolge arbeiten, enden sie auch im selben Zustand.

Dafür braucht man

- deterministische Zustandsübergänge
- gleiche Reihenfolge überall
- typischerweise **FIFO-Total-Order Broadcast**

Anwendungen

- replizierte Datenbanken
- verteilte Transaktionen

- Blockchains / Distributed Ledger

Nachteil

- Updates können nicht sofort lokal angewendet werden
 - man muss auf die Broadcast-Auslieferung warten
-

10. Leader-basierte Replikation

Grundidee

Ein **Leader** nimmt alle Schreiboperationen an:

1. Client sendet Update an Leader
2. Leader schreibt es in sein Transaktionslog
3. Leader repliziert es an Follower
4. Follower führen es in derselben Reihenfolge aus

Vorteil:

- einfache globale Ordnung

Problem:

- **Single Point of Failure**
 - wenn der Leader ausfällt, braucht man einen neuen
-

11. Konsens

Was ist Konsens?

Konsens bedeutet:

Eine Gruppe von Knoten soll sich auf **denselben Wert** oder **dieselbe Sicht** einigen.

Im Kontext von Replikation heißt das oft:

- Welche Nachricht ist die nächste?
 - Welche Reihenfolge gilt?
 - Welcher Leader ist gültig?
-

Formale Anforderungen

Ein Konsensalgorithmus soll typischerweise erfüllen:

Terminierung

Jeder Prozess entscheidet irgendwann

Gültigkeit

Entschieden wird nur ein Wert, der auch vorgeschlagen wurde

Integrität

Kein Prozess entscheidet zweimal

Übereinkunft

Keine zwei Prozesse entscheiden unterschiedlich

12. FLP-Satz

Der **FLP-Satz** sagt:

In einem **rein asynchronen** verteilten System ist Konsens im Allgemeinen **nicht lösbar**, selbst wenn nur ein Knoten ausfallen darf.

Deshalb arbeiten praktische Algorithmen wie:

- Paxos
- Raft

nicht mit völlig asynchronen Modellen, sondern mit **partieller Synchronität** und **Timeouts**.

Wichtig:

- physische Uhren dienen dort nur für **Timeouts** und **Fehlererkennung**
 - die **Sicherheit** des Algorithmus hängt nicht von exakter Zeit ab
-

13. Leader-Wahl

Warum braucht man sie?

Viele Konsens- und Replikationsalgorithmen nutzen einen Leader, um Ordnung herzustellen.

Wenn der Leader ausfällt:

- muss ein neuer gewählt werden
 - aber es darf nicht gleichzeitig zwei gültige Leader geben (**Split Brain**)
-

Terms / Epochen

Raft und ähnliche Verfahren teilen die Zeit in **Terms** ein.

- Jeder Term hat höchstens einen Leader
- Bei jeder neuen Wahl wird der Term erhöht

Wichtig:

- Es kann in verschiedenen Terms unterschiedliche Leader geben
 - Deshalb muss ein Leader immer wieder prüfen, ob er noch gültig ist
-

14. Raft – Überblick

Raft ist ein Konsensalgorithmus, der bewusst einfacher verständlich sein soll als Paxos. Er implementiert einen replizierten Log für State Machine Replication.

Ziel

- replizierter Transaktionslog
- replizierte Zustandsmaschine
- Fortschritt, solange eine **Mehrheit** der Server funktioniert

Fehlermodell

Raft toleriert:

- Crash / Fail-Stop
- Recovery
- verzögerte oder verlorene Nachrichten

aber **keine byzantinischen Fehler**.

15. Zustände in Raft

Jeder Knoten ist immer in genau einem dieser Zustände:

Follower

- passiv
- reagiert nur auf RPCs

Candidate

- startet eine Wahl

Leader

- nimmt Client-Anfragen an
 - repliziert Logeinträge an Follower
-

16. Leader Election in Raft

Heartbeats und Timeouts

Follower erwarten regelmäßig Nachrichten vom Leader.

Wenn für eine gewisse Zeit nichts kommt:

- vermutet der Follower einen Leader-Ausfall
- wird Candidate
- startet eine neue Wahl

Typische Timeout-Größe laut Folien:

- etwa **100–500 ms**
-

Ablauf einer Wahl

Ein Candidate:

1. erhöht den Term
2. stimmt für sich selbst
3. sendet `VoteRequest` an andere Knoten
4. gewinnt bei Mehrheit
5. wird Leader und sendet Heartbeats

Falls keine Mehrheit zustande kommt:

- neuer Versuch nach zufälligem Timeout

Die Zufälligkeit hilft gegen **Split Votes**.

Wann stimmt ein Knoten zu?

Ein Knoten stimmt für einen Candidate nur, wenn dessen Log mindestens so aktuell ist wie das eigene.

Dazu werden mitgeschickt:

- Länge des Logs
- letzter Term im Log

Das verhindert, dass ein veralteter Candidate Leader wird.

17. Log-Replikation in Raft

Normale Operation

1. Client sendet Kommando an Leader
 2. Leader hängt es an sein Log
 3. Leader schickt `AppendEntries` an Follower
 4. Follower speichern den Eintrag
 5. Sobald eine Mehrheit bestätigt, gilt der Eintrag als **committed**
 6. Dann wird er auf die Zustandsmaschine angewendet
 7. Später committen auch die Follower diesen Eintrag
-

Heartbeats

`AppendEntries` kann auch **ohne neue Logeinträge** verschickt werden.

Dann dient es nur als:

- Lebenszeichen des Leaders
 - Synchronisationssignal
-

18. Struktur des Logs

Ein Logeintrag enthält:

- Index
- Term-ID
- Kommando / Operation

Das Log liegt auf persistentem Speicher und überlebt damit Server-Crashes.

19. Log-Konsistenz in Raft

Raft erzwingt starke Konsistenzregeln für Logs:

Wenn zwei Server am gleichen Index denselben Term haben, dann gilt:

- sie speichern dasselbe Kommando
 - alle vorherigen Einträge sind ebenfalls gleich
-

Konsistenzprüfung bei AppendEntries

Jeder `AppendEntries`-Request enthält Infos über den **vorherigen** Logeintrag.

Ein Follower akzeptiert neue Einträge nur, wenn:

- sein Log an dieser Stelle passt
- der vorherige Eintrag denselben Term hat

Passt das nicht:

- lehnt der Follower ab
 - der Leader probiert es mit einem kleineren Index erneut
-

Reparatur inkonsistenter Logs

Der Leader merkt sich pro Follower einen `nextIndex` :

- das ist der nächste Eintrag, den der Follower bekommen soll

Wenn Replikation fehlschlägt:

- `nextIndex` wird reduziert
- Leader probiert es weiter vorne erneut
- der Follower überschreibt inkonsistente Einträge
- alle nachfolgenden falschen Einträge werden gelöscht

So werden divergente Logs wieder auf denselben Stand gebracht.

20. Commit in Raft

Ein Logeintrag gilt als **committed**, wenn:

- ein **Quorum** ihn bestätigt hat
- und er zum **aktuellen Term** gehört

Danach:

- wird das Kommando an die Anwendung ausgeliefert
 - Follower erfahren den Commit-Stand in späteren Nachrichten und committen ebenfalls
-

21. Wichtige Eigenschaften von Raft

Safety

- höchstens ein Leader pro Term
- konsistente Logs
- committed Einträge bleiben erhalten

Liveness

- irgendwann gewinnt jemand die Wahl
- zufällige Election-Timeouts helfen dabei

Fortschritt

- solange eine Mehrheit der Knoten verfügbar ist
-

22. Koordinierungsdienste

Systeme wie **ZooKeeper** bieten verteilte Koordinierung als Service.

Typische Funktionen:

- Locks / Semaphore
- Warteschlangen
- Leader Election
- Gruppenmitgliedschaft

- Barriers
- Konfigurationsmanagement

Solche Systeme basieren intern ebenfalls auf Konsensverfahren.

23. Wo wird das gebraucht?

Konsens und Replikation braucht man u. a. in:

- NoSQL-Datenbanken
 - verteilten Message-Queues
 - Service-Discovery-Systemen
 - Big-Data-Systemen wie Spark/Storm in Verbindung mit Koordinierungsdiensten
-

24. Kurzfazit

Replikation

Mehrere Kopien derselben Daten auf mehreren Knoten.

Hauptnutzen

- Verfügbarkeit
- Fehlertoleranz
- Lastverteilung

Hauptproblem

Änderungen müssen konsistent über Replikas verteilt werden.

Typische Werkzeuge

- Idempotenz
- Zeitstempel
- Tombstones
- Anti-Entropy
- Quoren
- Read Repair

Stärkere Lösung

Mit Total-Order Broadcast oder Konsens kann man replizierte Zustandsmaschinen bauen.

Konsens

Alle Knoten einigen sich auf denselben Wert bzw. dieselbe Reihenfolge.

Raft

Ein gut verständlicher Konsensalgorithmus mit:

- Leader Election
 - repliziertem Log
 - Quorum-basiertem Commit
 - Reparatur inkonsistenter Logs
-

25. Merksätze

- **Replikation erhöht Verfügbarkeit, macht Konsistenz aber schwieriger.**
- **Idempotenz ist zentral, wenn Requests wiederholt werden.**
- **Quoren ermöglichen Konsistenz trotz Ausfällen.**
- **State Machine Replication braucht überall dieselbe Reihenfolge.**
- **Konsens und Total Order Broadcast sind eng verwandt.**
- **In rein asynchronen Systemen ist Konsens grundsätzlich problematisch.**
- **Raft löst das praktisch mit Leader, Terms, Quoren und repliziertem Log.**